6.0 Static Techniques

(6 – 7 pm)

1

Terms you should know



Source: MSTB/GTB v1.0

2

Self evaluation

- How many 'F's can you find in the following text ?

```
FINISHED FILES ARE THE RE-
SULT OF YEARS OF SCIENTIF-
IC STUDY COMBINED WITH THE
EXPERIENCE OF YEARS
```

Source: MSTB/GTB v1.0

Answer!

- There are six!
- How many did you count ?
- To find 3 is normal, to find 6 is very good!
- "To err is human" ☺

```
FINISHED FILES ARE THE RE-
SULT OF YEARS OF SCIENTIF-
IC STUDY COMBINED WITH THE
EXPERIENCE OF YEARS
```

Source: MSTB/GTB v1.0

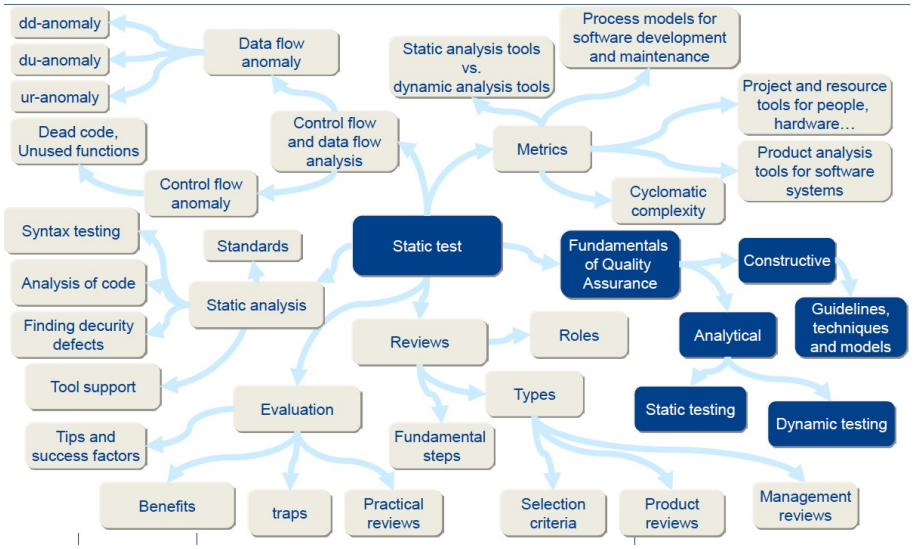
Sub-topics

- Fundamentals
- Reviews
- Static Analysis
- Control and data flow analysis
- Metrics

Source: MSTB/GTB v1.0

5

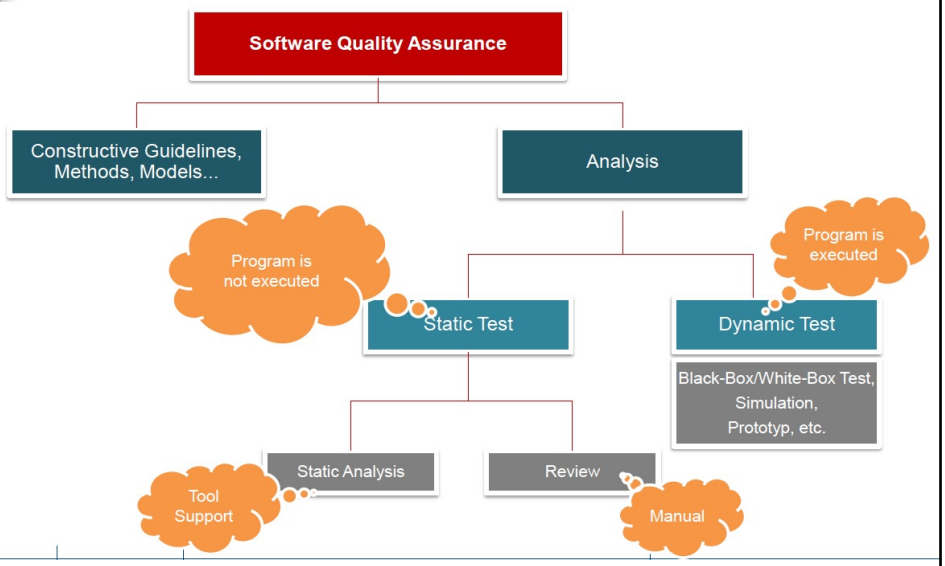
The big picture



Source: MSTB/GTB v1.0

6

SQA & Static Test



Source: MSTB/GTB v1.0

7

Static Test vs Dynamic Test

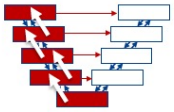
- In the development process, different products (artefacts) are created, e.g.
  - Informal Text
  - Models
  - Formal Text
  - Program code
- Programs are static descriptions of dynamic processes (computations)
- Dynamic tests check resulting processes via **interpretation** of a test object's description
- In a static test, the test object is not **executed**, rather only tested **as it is**
- Compared to dynamic testing, static techniques generally find the defects which are the causes of failures, rather than the failures themselves

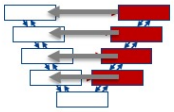
Source: MSTB/GTB v1.0

8

Static Test vs Dynamic Test in Software Lifecycle

- Static tests: Testing of a component or system at requirements or implementation level without execution of any software, e.g. reviews or static code analysis. [ISTQB-Glossary]
- In a static test, the test object will be verified against a document of a prior construction phase; in case of informal documents via review, in case of formal documents (e.g. program code) via a static analysis.
- Dynamic tests: Testing that involves the execution of the software of the component or system [ISTQB-Glossary]
- Documentation relating to test objects is created in a construction phase and used as a test basis (validation)





Source: MSTB/GTB v1.0

Static Test

- Analysis of a test object (mostly documents) via intensive observation
- Objective: Detection of faults/defects in an artefact
  - Deviations from standards, requirement defects, design defects, insufficient maintainability and incorrect interface specifications
  - Check the deviation from project test plans
  - Results of the analysis can be further used to optimize the development process
- Basic idea: Defect prevention!
  - Defects and incidents are detected as soon as possible, before they have an effect in the further software development and possibly require extensive correction and improvements

Source: MSTB/GTB v1.0

Manual vs Automated Testing

- Observation of the test object manually or via tool support
- Reviews
  - Manual testing by one or more people
  - Human analysis and thinking capabilities are used to test and evaluate work products
  - Any software work product can be reviewed (e.g requirement specifications, design specifications, code, test plans, test specifications, test cases, test scripts or user guides)
- Static Analysis
  - Automated testing by means of tool support against predefined rules (e.g. coding rules)
  - Used only for documents with a formal structure (e.g. program code, UML diagrams)

Source: MSTB/GTB v1.0

Manual Testing Techniques

- Non-tool-supported static testing use human analysis and thinking capabilities to test and evaluate complex work products
- This is performed via intensive reading and consideration of the documents to be tested
- Different approaches exist to statically test the documents.
- They differ in
  - the intensity of the analysis
  - the resources needed (people, time)
  - the desired goals

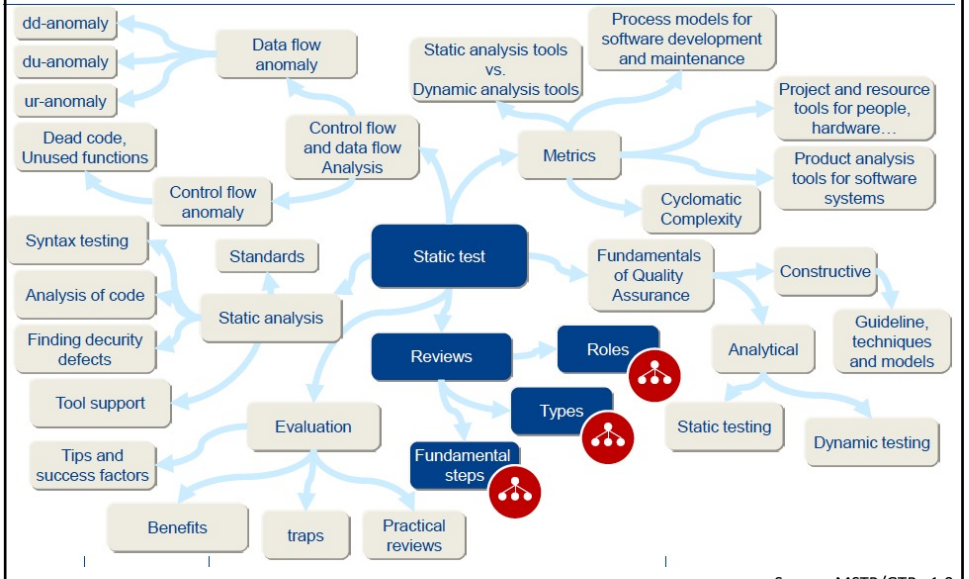
Source: MSTB/GTB v1.0

Sub-topics

- Fundamentals
- Reviews
- Static Analysis
- Control and data flow analysis
- Metrics

Source: MSTB/GTB v1.0

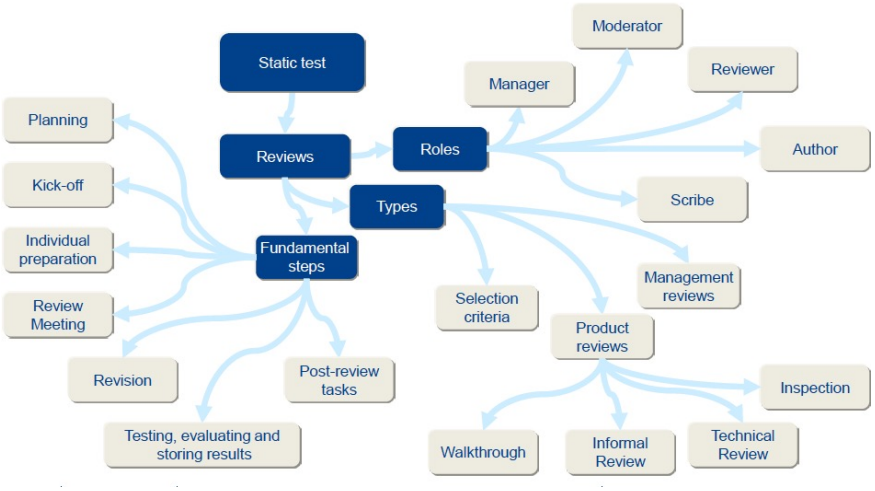
13



Source: MSTB/GTB v1.0

14

Zooming in



Source: MSTB/GTB v1.0

15

Reviews

- In general, a review is
  - an overview of something
  - a survey
  - a look back over past events
- "Review" is a term used to describe a particular way of testing software artefacts
- Often, reviews are the only possibility to test the semantics of such artefacts efficiently
- The term "review" is also a general term for a number of specific types of review. These will be covered in a later section of this chapter.

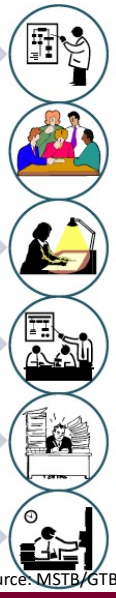
Source: MSTB/GTB v1.0

16

Fundamental Activities

With a formal review, there are 6 basic steps to execute:

1. Planning
2. Kick-Off
3. Individual Preparation
4. Examination
5. Rework
6. Follow Up



Source: MSTB/GTB v1.0

17

Reviews: Planning (1)



- The management decides which documents should be reviewed
- The estimated expenses for the separate reviews is to be considered in the project planning
- The entry criteria for the execution of the review are confirmed (especially with more formal reviews)
- The exit criteria for the completion of the review are defined (especially with more formal reviews)
- Furthermore, testing criteria are confirmed

1. Planning
2. Kick-Off
3. Individual Preparation
4. Examination
5. Rework
6. Follow Up

Source: MSTB/GTB v1.0

18

Reviews: Planning (2)



- The manager chooses professionally qualified members and puts together a review team with specific roles (see later)
- The manager assures in cooperation with the author of the test object, that it has a »reviewable« state, i.e. the document is to a certain degree finalized and has reached a satisfactory maturity
- Re-examination if the entry criteria for the execution of the review are satisfied
- If a Kick-Off meeting takes place, time and venue have to be fixed

1. **Planning**
2. Kick-Off
3. Individual Preparation
4. Examination
5. Rework
6. Follow Up

Source: MSTB/GTB v1.0

19

Reviews: Kick-Off (1)



- All people who are involved in the review are provided with the necessary information
- Written invitation or spontaneous first meeting of the review team to inform about the reason and the objectives of the review.
- If the involved people are not fully knowledgeable of the document to be reviewed, it can be briefly introduced and its relevance to the application area can be presented.

1. Planning
2. **Kick-Off**
3. Individual Preparation
4. Examination
5. Rework
6. Follow Up

Source: MSTB/GTB v1.0

20

Reviews: Kick-Off (2)



- In addition to the document which is to be reviewed, the involved people are provided with further data:
- Other documents are needed to help decide if a deviation, a defect, or a correct description of the issue is present. These other documents could include, for example, requirements specification, guidelines, standards
- Checklist are very useful, as they support a structured review process and enable common defects to be found more easily (see example on next two pages).



Source: MSTB/GTB v1.0

21

Reviews: Checklist Example(1)



Review the changed component requirements of the system design to ensure completeness:



- Has every changed or newly designed class of the system design been represented at the code level ?
- Do all the attributes exist of a changed or newly designed class of a system design in the code (methods with parameters and return values, association, aggregation, inheritance)?
- If the changed or newly designed class inherits from an “Observer” class, does the class attach and detach itself afterwards from the observed subject ?

Source: Software Engineering Working Group
Prof. Dr. H. D. Rombach, University of Kaiserslautern
Checklist for inspection of development (German)
http://wwwagse.informatik.uni-kl.de/teaching/se1lab/ss2002/Phase4/Verifikation_ChecklisteSKD.pdf.

Source: MSTB/GTB v1.0

22

Reviews: Checklist Example (2)



- Was every state diagram from the sytem design turned into code? Is every state, state transition (with appropriate actions, if they exist), do-activities and entry-exit actions represented in code ?
- Was a state of type OR assigned to an enumeration type (see guidelines for the derivation of Java code from OMT/UML state diagrams)?
- Was for each state of type BASIC an IF- statement defined ?
- Were the sub-conditions of a type OR state implemented by means of a SWITCH statement?
- Were event-triggered transitions and internal actions implemented by means of a partial state diagram?
- Does the class import the classes mentioned in the design under "imports"?
- ...



Source: MSTB/GTB v1.0

23

Reviews: Individual Preparation



- Each reviewer studies the document intensively and uses the data provided
- Use of checklists and other documents to support the task is recommended
- Any potential defects, questions or comments should be noted
- The review team prepares themselves for the review meeting (examination)



Source: MSTB/GTB v1.0

24

Reviews: Examination (1)



- Conducted by the moderator
- Generally of fixed duration (max. 2 hours)
- If needed, a further examination can be called for (on the next day at the earliest)
- Objective: Evaluation of the document to be reviewed with regard to the adherence and implementation of relevant rules and guidelines
- The evaluation of the specific findings and the over-all conclusion should be agreed upon by all reviewers



Source: MSTB/GTB v1.0

25

Reviews: Examination (2)



- The moderator has the right to cancel or discontinue an examination if one or more experts (reviewers) are absent or are not appropriately prepared
- The review document is to be scrutinized and not the author!
  - Reviewers must pay attention to expressions and wording
  - The author should not try to defend defects found
  - To ensure this, reviewers should not “attack“ the author in a way that forces a defensive response
  - A justification the author‘s decision will be viewed partly as appropriate and helpful
- The moderator shall not take a reviewer role at the same time
- No general matters of style and grammar (outside of the guidelines) are to be discussed

Source: MSTB/GTB v1.0

26

Reviews: Examination (3)



- Development and discussion of solutions is not the task of the review team
- Every reviewer must have the opportunity to present his/her comments appropriately
- The consensus of the reviewer about a comment is to be continuously recorded by the scribe
- Defects are not to be recorded as instructions to the author
- Instructions in the form of additional, specific suggestions for correction might be viewed to a certain degree as useful and helpful for the improvement of the quality

Source: MSTB/GTB v1.0

27

Reviews: Examination (4)



Prioritizing of defects found. For example

- Critical Defect
  Test object cannot be used until corrected.The defect must be corrected before the next release
- Major Defect
  Usability of the test object is limited. The defect should be corrected before the next release
- Minor Defect
  A small issue, e.g. spelling mistakes or flaws in expression which do not affect usability
- Good
  free of flaws, no further changes to be done

Source: MSTB/GTB v1.0

28

Reviews: Examination (5)



- Review team gives recommendations about the adoption of the test object:
  - Accept without changes
  - Accept with changes – no further review needed
  - Reject – further review or other test required
- The review report contains a list of all defects, which have been discussed in the examination and, in addition, to that
  - Information about the test object and the data used
  - People involved and their roles
  - A short summary of the important points
  - Result of the examination with the recommendation of the review team
- At the end, all members of the examination approve and sign the review report

Source: MSTB/GTB v1.0

Reviews: Report

- Example of a single paged summary

Review	Summary of results				
Project					
Author:					
Document:			Version:		
Type of review					
Planning					
Responsible for performing the Review					(1)
Document checked:	Yes☐		Date		No ☐
Document checked:	Yes☐		Date		No ☐
Preparation					
Reviewer 1			Duration	h	(2)
Reviewer 2			Duration	h	(2)
Reviewer 3			Duration	h	(2)
Reviewer 4			Duration	h	(2)
Reviewer 5			Duration	h	(2)
Reviewer 6			Duration	h	(2)
Review-Meeting					
Moderator:					
Date		Participants	Duration	h	
			Total effort:	h	(3)
Results					
Severe defects		Renewed inspection needed: Yes☐		No ☐	
Minor defects:					
Clarifications		Rework complete by:			
Proposals					
Document defect		Total effort:			=(2)+(3)
Conclusion					
Result OK, ework completed.					
Datum:		Signature (1):			

Source: MSTB/GTB v1.0

Reviews: Rework



- Correcting the discovered defects and recording the new status
- Generally by the author
- Depending on the ease of understanding of the defects, it might be useful to discuss with the reviewer again

1. Planning
2. Kick-Off
3. Individual Preparation
4. Examination
5. **Rework**
6. Follow Up

Source: MSTB/GTB v1.0

31

Reviews: Follow Up



- Ensuring that the review has been executed acccording to the guideline
- Checking that defects have been removed by means of rework
- Gathering metrics
- Checking on exit criteria (for more formal review types)
- Manager decides if the recommendation of the reviewers will be followed or makes a different decision, for which he/she then bears the sole responsibility
- If the result of the first review wasn't acceptable, and another review is to be done, the second review should be done according to the described process (although often shortened)

1. Planning
2. Kick-Off
3. Individual Preparation
4. Examination
5. Rework
6. **Follow up**

Source: MSTB/GTB v1.0

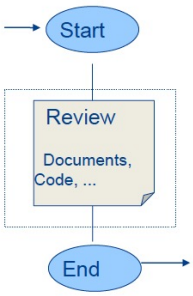
32

Reviews: Review process

Input

- Document/object to be examined
- Adaption of the review process
- Review-Check list
- Project data

Process



Output

- Defects, incidents
- Review check list (filled out)
- Results, task sheet

Metrics, for example:

- Amount of defects
- Amount of confirmed changes
- Complexity of review

Source: MSTB/GTB v1.0

Roles in the review process (1)



- Manager
  - Decides on the execution of reviews
  - Chooses the testing objects and the documents to be used
  - Provides the necessary resources and allocates the review team
  - Determines if the review objectives have been met
  - Decides about the next steps after the examination
- Moderator
  - Leads the review and is the person upon whom the success of the review rests
  - Often responsible for the planning activities
  - Has to mediate between the various points of views and pays attention to any subtle comments
  - Is not to be prejudiced and is not permitted to express his/her own opinion regarding the testing object

Source: MSTB/GTB v1.0

Roles in the Review Process(2)



- Author
  - Writer of the document which is to be reviewed
  - If there are several authors, one responsible writer should be named, who will take over the overall role
  - It is important, that the author does not consider the expressed views as a criticism of his own personality, but that it is exclusively about the improvement of the quality of the reviewed document
- Reviewer (Inspector)
  - Several (maximum five) experts, who take part in the examination after the necessary preparation
  - Ensure that parts of document, which are found to be ‚good', are also being labeled as such
  - Will clearly mark defective parts of the review object so that their removal is possible
  - Reviewers should be chosen to represent different perspectives and roles in the review process and can take part in all the examinations

Source: MSTB/GTB v1.0

35

Roles in the Review Process(3)



- Scribe
  - Documents all the issues, problems, and open points that were identified during the meeting
  - Has to formulate concisely and precisely and write legibly
  - Writes up the discussions taking place at the meeting so that they can't be corrupted
  - The roles of scribe and moderator should not be taken by one and the same person
  - For practical reasons, it might be useful to let the author to also be the scribe. The author knows exactly what to note down and with what level of precision. This ensures that sufficient information is available to make the changes later.

Source: MSTB/GTB v1.0

36

Reviews: Roles (Summary)



Manager	Decides on the execution of reviews, allocates time in project schedules and determines if the review objectives have been met.
Moderator	The person who leads the review of the document(s) including planning of the review, running the examination and following up after the meeting. If necessary, the moderator may mediate between the various points of view, and is often the person upon whom the success of the review rests.
Author	The writer or person with chief responsibility for the document(s) to be reviewed.
Reviewer	Individuals with a specific technical or business background (also called checkers or inspectors) who, after the necessary preparation, identify and describe what they have found (e.g. Defects).
Scribe	Documents all the issues, problems and open points that were identified during the examination



Source: MSTB/GTB v1.0

37

Types of reviews

- Reviews vary from quite informal to very formal (that means, well structured and organized)
- The degree of formality of the review process to be specifically applied depends on factors such as
  - Maturity of the development process
  - Legal or regulatory requirements
  - The need for evidence that the review was performed
- The way in which a review is conducted depends on the objectives of the review, e.g.
  - The finding of issues and defects,
  - Spreading knowledge and understanding
  - A discussion with agreements by consensus

Source: MSTB/GTB v1.0

38

Reviews: Types according to the examined object

- Management Reviews
  - Analyse project plans and the development project in itself
- Product Reviews
  - Refer to products or partial products which are being created during a development process
- Product reviews are conducted using one of the following types of review:
  - Informal Review
  - Walkthrough
  - Technical Review
  - Inspection

Source: MSTB/GTB v1.0

Management Reviews: Main Characteristics

- No single documents will be reviewed
  - The project as a whole is in focus
  - The project status is evaluated according to technical, economical, scheduling and management aspects
- As preparation, talks on the chosen topics can be prepared
  - Basis for the review are, among others, the project documentation and the protocols of the reviews performed on individual documents or products up to that moment
  - Prepared presentations will be given in the examination
- Management-Reviews
  - Can take more than 2 hours, they can even last up to 2 days
  - Are often performed when a milestone in the project planning has been reached, when a main phase of the software development process is to be concluded or as a post-mortem analysis, so as to learn from the terminated project

Source: MSTB/GTB v1.0

Informal Review

- Informal review during a developmental activity
- Mostly a single review with no formal process
- Can be optionally documented
  - *Desktop Test* of an own program or (better) the program of someone else
  - *Peer-Principle* -can be integrated into Pair Programming (XP) or a technical lead programmer reviewing design and code
- Varies in usefulness depending on the reviewers
- Inexpensive way to get some benefit

Source: MSTB/GTB v1.0

Walkthrough (1)

- Main purposes: Learning, gaining understanding, finding defects
- Definition: A step-by-step presentation by the author of a document in order to gather information and to establish a common understanding of its content [Freedman and Weinberg] and [ISTQB Glossary].
- Keypoint is the examination (often *open end*)
- In comparison with the other types of review, the preparation has the least complexity. It can be partly even waived.
- During the examination, the author often takes over the role of the moderator (but not the role of the scribe).
- The author presents the review object to the reviewers
- Since the author conducts the examination, he/she can influence its course strongly. This can be a disadvantage if the author does not permit discussion of critical points of the review object with the required intensity or doesn‘t even permit the discussion at all.

Source: MSTB/GTB v1.0

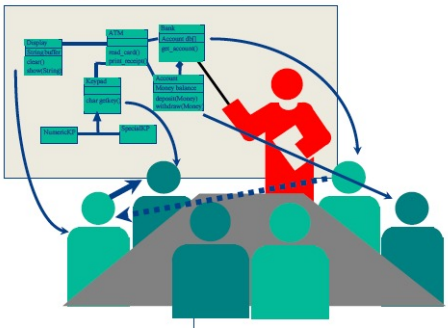
Walkthrough (2)

- Mostly takes the form of typical cases of application (User situations, dry runs, scenarios) in peer-group application
  - Separate test cases can also be walked through
  - Reviewer tries mostly by means of spontaneous questions to uncover possible defects and problems.
- Appropriate for small developer teams and reviews of “non-critical“ documents
- Little effort needed, as pre- and post-processing is limited or not required at all
- In practice, list of findings and reports can vary from informal to very formal
- The author is responsible for the post-processing. There is no post review control

Source: MSTB/GTB v1.0

Specific case: design walkthrough

- Each reviewer takes over the responsibility for one or more components of the design
- According to specific scenarios, interactions between the components can be re-enacted
  - Interfaces and parameters have to be observed
  - Which component has which information at what stage?



Source: MSTB/GTB v1.0

Technical Review (1)

- Main purposes:
  - Discussing, decision making, evaluating alternatives, finding defects, solving technical problems
  - Checking conformity of the review object to specifications, adherence to regulations, suitability for the intended purpose.
- Definitions:
  - A peer group discussion activity that focuses on achieving consensus on the technical approach to be taken [Gilb and Graham, IEEE 1028] and [ISTQB Glossary].
  - Documented, defined process for detecting defects that includes peers and professional experts
- Can be done as a *Peer Review* without the participation of the management, and vary in practice from informal to very formal
- In case of high formality, entry- and exit-criteria may be defined
- It is not the responsibility of the technical review team to decide about the consequences of the review results

Source: MSTB/GTB v1.0

45

Technical Review (2)

- Optional items
  - Using checklists
  - Producing a review report or a list of findings
  - Participation of the management
- Substantial time and effort needed in preparation phase
  - Reviewers test the object according to fixed testing criteria
  - Reviewer comments about each testing criteria are to be recorded in writing and handed to the moderator in advance
- During the examination
  - Ideally led by a trained moderator (not the author)
  - Only relevant comments should be discussed
  - Scribe documents all comments and produces a final documentation of the verdict
- The moderator prioritizes reviewer comments
- Review result must be agreed by consensus by the review team

Source: MSTB/GTB v1.0

46

Inspection (1)

- Main purpose: Finding defects (or even causes for defects)!
- Predefined, formal process, based on rules
  - The most formal type of review
  - Objectives defined at the planning stage
  - Can also be used to improve processes
  - Each person has a defined role
  - Formal follow-up process
- Usually conducted as peer examination
- Limited amount of aspects to be covered per reviewer (Inspector)
- Criteria for each aspect mostly defined in checklists, with specified entry- and exit-criteria for separate testing steps
- Includes metrics to evaluate the product
- Results of the inspection should not only evaluate the quality of the reviewed document but also an evaluation of the development- and inspection processes themselves

Source: MSTB/GTB v1.0

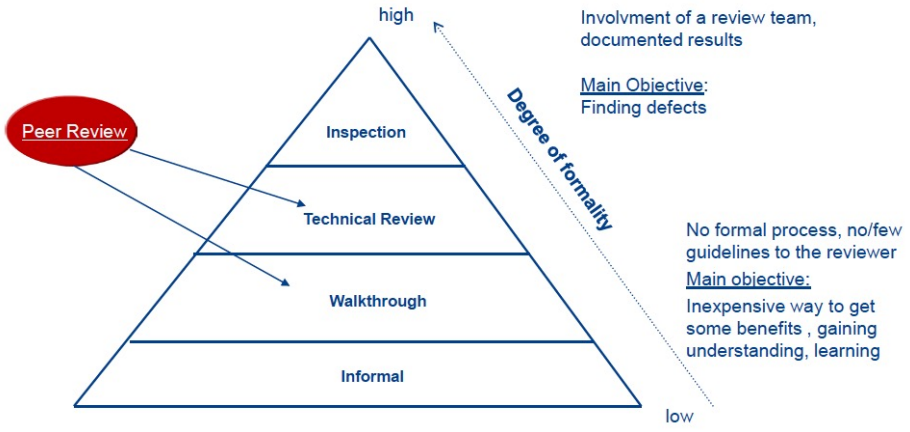
Inspection (2)

- Examination is led by a trained moderator (not the author)
  - Starts with a short introduction. A reviewer presents concisely and in a logical manner the contents of the review object; if deemed useful, passages can also be read out
  - Reviewers state defects found and refer to individual checklist points
  - Author may answer any questions addressed to him, but is otherwise passive
  - Moderator intervenes if any discussion develops which is not directed at finding defects
  - All established (and deferred) disagreements and defects are recorded by the scribe
  - At the end of the examination, recorded defects may be presented and checked for completeness by all involved

Source: MSTB/GTB v1.0

Types of Review (Summary)

Types of Reviews accoding to IEEE 1028



Source: MSTB/GTB v1.0

49

Reviews: Selection Criteria (1)

- **Does the degree of formality of the testing object allow tool based analysis ?**
  - Yes ⇒ First of all use a tool
  - No ⇒ Use one of the types of review
- **How is the required degree of formality of the testing result ?**
  - Extensive formal documentation
    ⇒ Inspection or technical review
  - Undocumented implementation of the testing results
    ⇒ Walkthrough or informal review
- **Is expert knowledge of several areas required ?**
  - Yes ⇒ Technical review
  - No ⇒ Walkthrough or inspection
- **How much expert knowledge about the review object do the reviewers need ?**
  - Much ⇒ First of all, conduct walkthrough
  - Little ⇒ Start directly with an inspection or a technical review

Source: MSTB/GTB v1.0

50

Reviews: Selection Criteria (2)

- **Scheduling of appointments easy or difficult ?**
  (To call five to seven experts to one or several appointments can be very challenging)
  - Easy ⇒ Inspection or technical review
  - Difficult ⇒ Walkthrough or informal review
- **Is the complexity of preparation appropriate to the to be expected result ?**
  - Yes ⇒ Inspection or technical review
  - No ⇒ Walkthrough or informal review
- **How extensive is the availablity of the chosen reviewers ?**
  - High ⇒ Inspection or technical review
  - Low ⇒ Walkthrough
    (or: cancel the review, as there is mostly no beneficial result if the availability is limited!)

Source: MSTB/GTB v1.0

51

Reviews: Benefit (1)

- Correction of defects costs resources – if defects can be recognized and corrected at an early stage, it leads to an increased development productivity as less resources have to be used for defect recognition and correction, and these resources are therefore available for development
- It leads often to shorter development timescales
- If defects are detected and corrected in an early stage, it leads to a reduction of the effort needed for dynamic test, as less defects are left in the test object
- Fewer defects in a running system
- The responsibility of the the quality of an examined review object lies with the team
- Mutual learning and possible process improvements are further positive effects

Source: MSTB/GTB v1.0

52

Reviews: Benefits (2)

- Because of the low amount of defects and deviations, a reduction of costs can be expected in the lifetime of the review object
  - Inaccuracy of customer requirements can often be discovered by means of reviews and settled immediately
  - The amount of requested changes after adoption of the software system can be reduced
- As the review is being done in a team, it leads to a knowledge exchange for everyone involved, which improves the working processes of each participant and therefore also leads to an improved quality of later products
- As several people are involved in a review, a clear and easily understandable presentation of issues is required
- Often, the need to define issues clearly leads the author to insights that were not previously available

Source: MSTB/GTB v1.0

Inspections: costs and savings!

- Costs of inspections are estimated to cover 10%-15% of the development budget
- Savings between 14% and 25% are possible, already including the extra costs of reviews
- When used consequently, up to 70% of the defects can be found in a document
- Reduction of defect costs up to 75%

Metrics taken from
Gilb, T.; Graham, D.: Software Inspections. Addison-Wesley, 1996
and
Bush, M.: Software Quality: The use of formal inspections at the Jet Propulsion Laboratory. In: Proc. 12th ICSE, IEEE 1990, 196-199
and
Frühauf, K.; Ludewig, J,; Sandmayr, H.: Software testing (German). Verlag der Fachvereine, Zürich, 4. Issue. 2000

Source: MSTB/GTB v1.0

Reviews: Risk factors (1)

- Not enough staff available or not enough with the required training
  - Conduct training or borrow appropriate staff from external companies (especially the moderator, as he/she has to possess psychological and project-specific capabilities)
- Not enough time (incorrect estimation when planning resources)
  - Possibly, a less extensive type of review can solve the problem
- Poor preparation
  - Choose different reviewers
  - If reviewers are not convinced of the importance of the review and its high level of effectiveness regarding quality improvements to the examined documents, this is to be emphasized with adequate numeric data

Source: MSTB/GTB v1.0

Reviews: Risk factors (2)

- Non-existant or confused objectives
  - Finding defects? Improving the process? Blaming people?
  - Confirm the objectives which are to be achieved with this review when planning it (i.e. prior to conducting the review!)
- Lacking or inadequate documentation
  - Check in advance, whether all needed documents are available and in the depth needed. Only if this is the case can the review be done
- No management support (unfortunately very common in practice)
  - The review process is not going to be successful if the required resources are not made available and the achieved results are not used for the improvement of the process

Source: MSTB/GTB v1.0

Reviews in Practice

- Reviews are not accepted as a method
- Implementation is not very systematic
- Hardly any specific training for software reviews
- Special procedures like code inspections are used considerably less in small companies than in large ones (less than 50 staff in software development)
- Only few of the surveyed people stated that they evaluate the quality of reviews with accompanying measurements (50% don't collect any data at all, 20% cost data, 20% defect data).

Results of a survey of the Fraunhofer Institut IESE
under the ViSEK Kompetenzzentrums in summer 2002
121 participants from companies of all areas with own sw development

http: //www.software-kompetenz.de

Source: MSTB/GTB v1.0

Reviews : Tips (1)

- Analyse documents with a formal structure, first of all, using a review tool. This can check many aspects and identify defects and incidents which then don't have to be checked in a manual review
- A review examination is possible if all involved people are well prepared
- Evaluation of each review examination and its results are to be used to improve the review process and to adjust the supporting documents used (guidelines, checklists) to a specific environment and keep them up to date
- Accept found defects positively and discuss them objectively

Source: MSTB/GTB v1.0

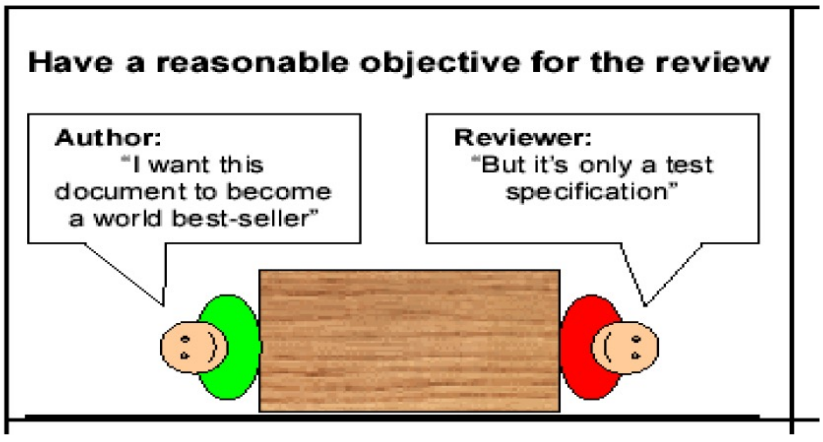
Reviews: Tips (2)

- Handle the questions of involved people constructively, keep in mind psychological aspects. For example, make the review a positive experience for the author
- Re-occurring or often appearing defects pinpoint deficits in the software development process or a lack of expertise from the staff involved
  - Planning and implementing needed improvements of the development process
  - Compensate for the lack of expertise with training
- IEEE Std 1028-2008 is the "IEEE Standard for Software Reviews"
  - Adhere to this standard!

Source: MSTB/GTB v1.0

Reviews: Success Factors

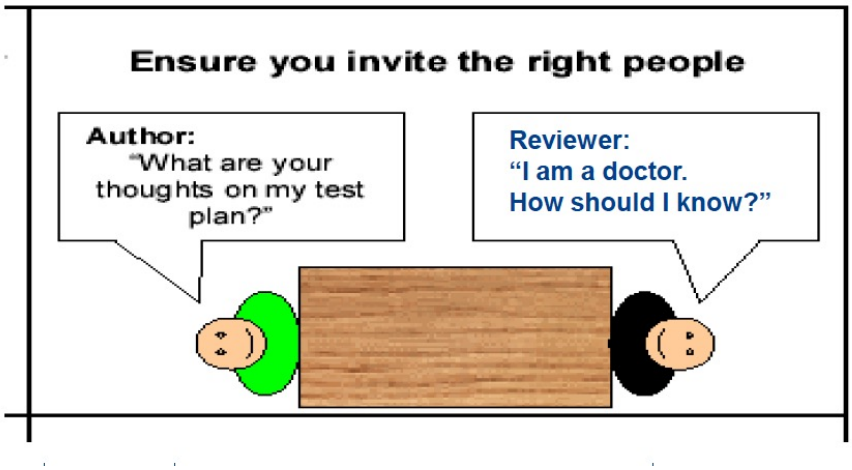
- **Each review has clear predefined objectives.**



Source: MSTB/GTB v1.0

Reviews: Success Factors

- **The right people for the review objectives are involved**



Source: MSTB/GTB v1.0

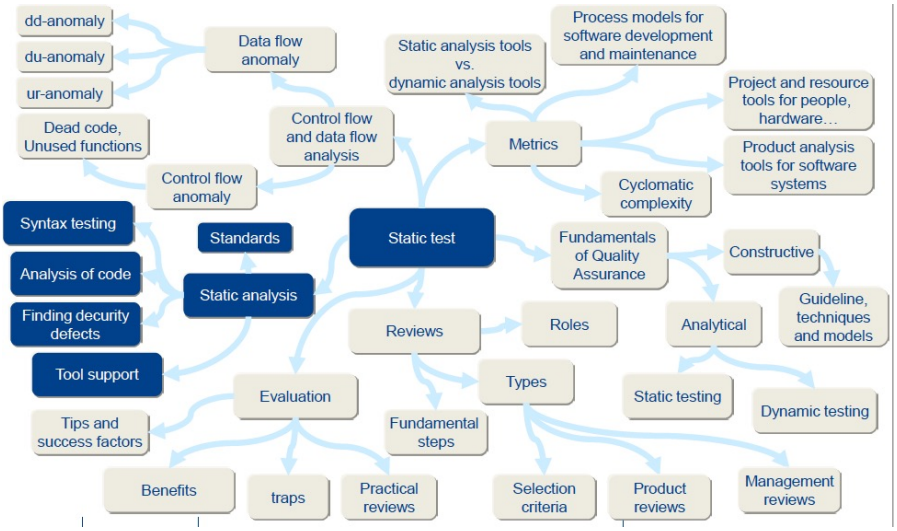
61

Sub-topics

- Fundamentals
- Reviews
- Static Analysis
- Control and data flow analysis
- Metrics

Source: MSTB/GTB v1.0

62



Source: MSTB/GTB v1.0

63

Static Analysis

- Objective: To find defects or error-prone areas in a document
  - The term »static analysis« indicates that this type of testing does not include the execution of the test object (program code)
- Example
  - Spell-checker programs as a kind of static analyser, that detects defects in documents (spelling and to a certain degree grammar mistakes), and thereby plays its part in quality improvement
  - Compiler carries out an analysis of program code, by checking against syntax violations of the programming language
  - Specialised tools which check use of programming standards
- Further Objective:
  - Collect measures or metrics to enable a quality evaluation to be performed

Source: MSTB/GTB v1.0

64

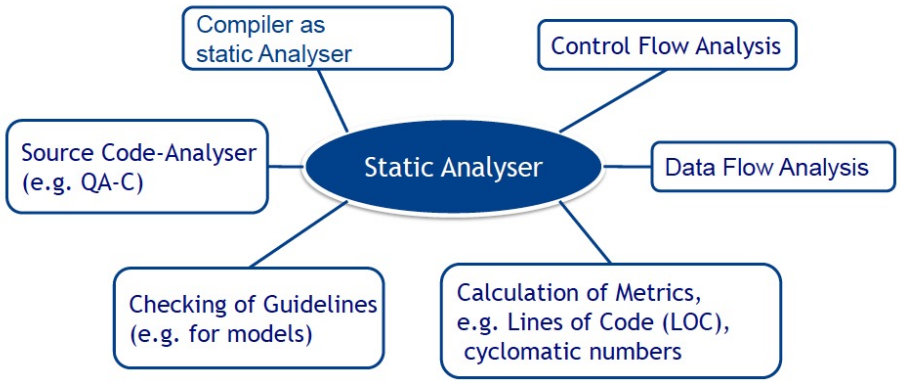
Static Analysis and Tool Support

- Static analysis is only meaningful with tool support!
- The document which is to be analysed must adhere to a formal structure (e.g. programming language syntax) to enable analysis to be performed by a tool
- Other documents, that adhere to a formal structure, are for example:
  - Technical specifications
  - Software architecture
  - Software designs
- With the exception of spelling and grammar checks, informal text can only be analysed by means of Artificial Intelligence (AI)
  - Linguistic semantic analysis is a current matter of research
  - The introduction of a »standard language« can enable simple analysis

Source: MSTB/GTB v1.0

65

Elements of a Tool-based Static Analysis



Source: MSTB/GTB v1.0

66

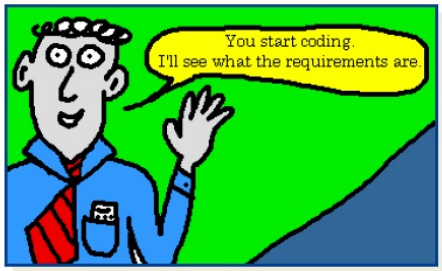
Static Analysis and Reviews

- They are closely related to each other
  - If a static analysis is being conducted prior to the review of a formal document, a number of defects and deviations can be identified and the amount of aspects to be considered in a review is substantially lower
  - As static analysis is conducted with tool support, the complexity is substantially less than with a review
- Therefore:
  1. Static Analysis, and then
  2. Review

Source: MSTB/GTB v1.0

Static Analysis in Practice

- Common practice in software development :



- Effect: Unfortunately, the program code is often the first (and only) formal document of software development, which can be subjected to a static analysis
- Question: Is that a problem ?

Source: MSTB/GTB v1.0

Static Analysis of Program Code

- All compilers conduct a static analysis of the program code, by checking the adherence of the syntax of the respective programming language
- Most of the compilers also provide additional information, which can be determined via static analysis
- Beside compilers, there are tools, called Analysers, which can be used for specific analysis purposes
- Examples of defects or error-prone situations, which can be detected by the static analysis of program code are:
  - Syntax violations
  - Convention and standard deviations
  - Security vulnerabilities
  - Control flow anomalies
  - Data flow anomalies

Source: MSTB/GTB v1.0

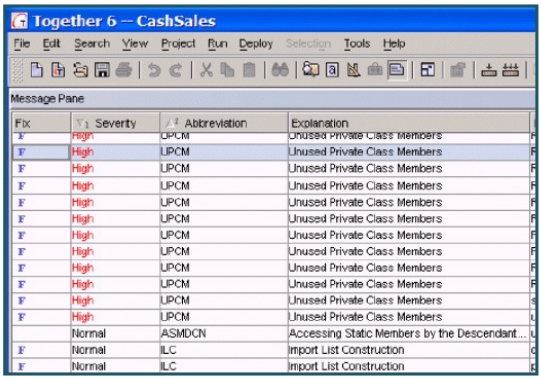
Syntax Testing (1)

- Syntax violations of the programming language will be reported as warnings or defects
- Many compilers conduct further checks, such as:
  - Usage of separate elements of the program (variables, functions, ...)
  - Testing of properly formatted usage of data and variables with strongly typed programming languages (C, C++, Java)
  - Detection of undeclared variables
  - Unreachable (dead) code
  - Defects in the over- and under-shooting of array boundaries
  - Checking of consistency of interfaces
  - Checking of the usage of all tags as jump instructions and jump targets

Source: MSTB/GTB v1.0

Syntax Testing (2)

- The results are normally presented as a list
- The results don't always directly indicate defects. Further checks are necessary



Source: MSTB/GTB v1.0

Adherence of Conventions & Standards

- The adherence of programming or design conventions by means of appropriate analysers can be checked
  - Needs less time and almost no staff resources
  - Furthermore, there is often another benefit given: If the programmers or the software designers are aware, that the program code or the design of a tool is being checked against the adherence to guidelines, their willingness to implement it, is substantially higher than without an automated testing
- Tip: Only allow such guidelines to be used in a project when testing tools are made available (other standards prove themselves quickly, in practice, to be only a bureaucratic burden)
- But: Static analysis tools may produce a large number of warning messages and hints, which need to be well-managed to allow the most effective use of the tool

Source: MSTB/GTB v1.0

Discovering of Security Vulnerabilities

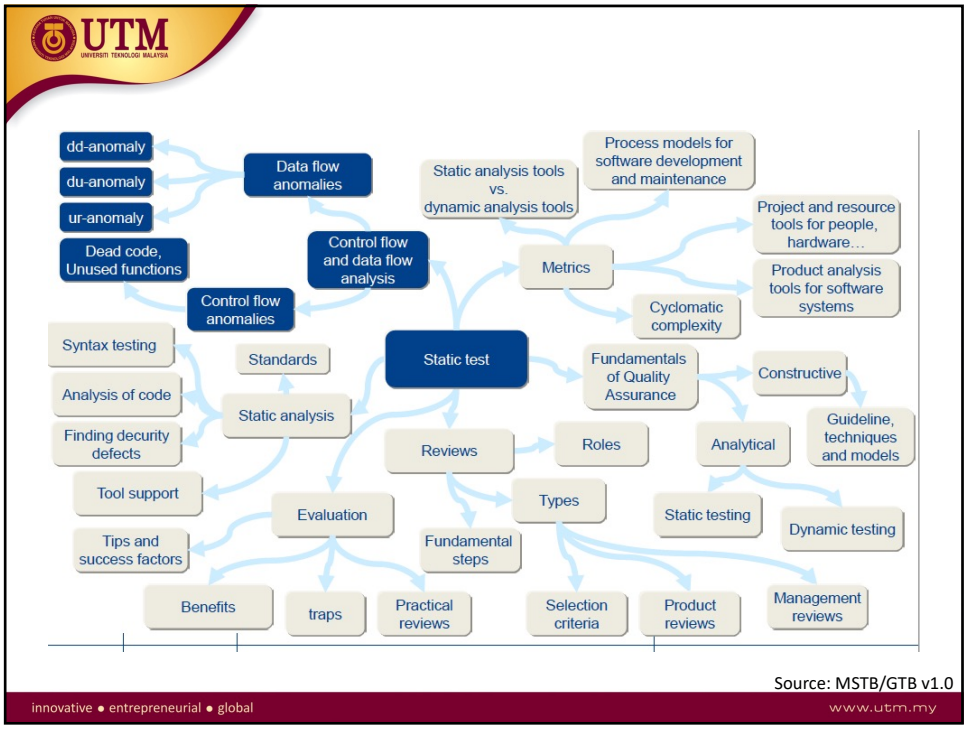
- Because of the usage of certain error-prone programming structures and the absence of necessary reviews, security vulnerabilities can appear
- Example:
  - No interception of input-buffer overflow
  - No testing of adherence to data limits at the entry
- Analysis tools can discover these defects, as they are mostly of a certain “structure“, which can be traced and analysed by tools

Source: MSTB/GTB v1.0

Sub-topics

- Fundamentals
- Reviews
- Static Analysis
- Control and data flow analysis
- Metrics

Source: MSTB/GTB v1.0



Source: MSTB/GTB v1.0

75

Control and Data Flow Analysis

- Searching for anomalies in the program code
- Anomalies: Irregularities, which might be a defect which lead to a failure, but do not have to lead to one.
  - anomaly: irregular, against the rule (acc. to dictionary)
- Not all defects and irregularities can be identified with static analysis: Some defects can only be identified as a failure while the program is running, and not before
  - If, for example, the divisor in a division is a variable, the variable could take the value zero, which might lead to a failure
  - Statically this defect is usually not detectable

Source: MSTB/GTB v1.0

76

Control Flow

- Control Flow: an abstract representation of all possible sequences which can be executed by a program
  - Sequence of statements executed while the program is running
  - Doesn't have to be in the sequence of the statements in the program code
- Is being determined by »controlling« statements
  - GOTO's
  - Conditional branches
  - Loops
- The logical value of conditions, e.g. In IF-statements in program code, governs the flow of control
  - If the condition evaluates to "true", then the program will be executed with the THEN-part of the IF–statement. In the opposite case ("false"), the ELSE part will be executed.
- Loops lead back to prior statements, leading to either repeated execution of the code within the loop or exiting the loop.

Source: MSTB/GTB v1.0

Control Flow Graph: Elements

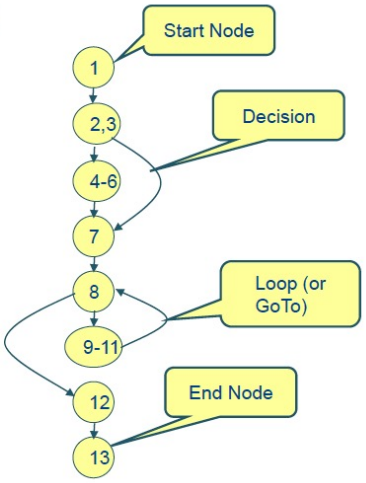
- Directed graph
- Nodes: Statement of the program
  - Linear sequences of statements (block, segment) can be presented as a single node
  - if the first statement of the sequence is executed, all further statements of the sequence will also be executed
- Edges: Possible execution sequences of the statements, represented by the nodes
  - Nodes with several outgoing edges represent the statements that control the flow
  - IF-statements and loop statements: Two outgoing edges
  - CASE-statements: several outgoing edges
- There is exactly one start node at the beginning of a program, or head of the method and exactly one end node at the end of the program or method

Source: MSTB/GTB v1.0

Example of a Control Flow Graph

```
1. public int euclid (int m, int n) {

2.    int r;
3.    if (n > m) {
4.       r = m;
5.       m = n;
6.       n = r;
      }
7.    r = m % n;
8.    while (r != 0) {
9.       m = n;
10.      n = r;
11.      r = m % n;
      }
12.   return n;
13. }
```



Source: MSTB/GTB v1.0

Control Flow Analysis

- The clarity of the control flow graph easily shows flows of a program
- If all or parts of the graph are unclear and the application flow is hard to trace, the program code should be reworked because complicated application control flows are often error prone
- Control Flow Anomalies: Statically detectable irregularities in the flow of the test object (e.g. statements not reachable)
  - Jumps out of or into loops
  - Program segments with several exits
- These anomalies don‘t have to be defects, but contradict the fundamentals of well-structured programming
- Requirement: Graph should not be produced manually but by means of a tool, which can guarantee an exact mapping exists between the program and graph

Source: MSTB/GTB v1.0

Control flow analysis

- Highlights:
  - nodes not accessible from start node
  - infinite loops
  - multiple entry to loops
  - whether code is well structured, i.e. reducible
  - whether code conforms to a flowchart grammar
  - any jumps to undefined labels
  - any labels not jumped to
  - cyclomatic complexity and other metrics

81

Unreachable Code

- Undesirable:
  - Occupies unnecessary memory
  - Causes unnecessary caching of instructions into the CPU instruction cache - which also decreases data locality.
  - From the perspective of program maintenance; time and effort may be spent maintaining and documenting a piece of code which is in fact unreachable, hence never executed.

82

Unreachable code (cont.)

```
int foo (int iX, int iY) {
     return iX + iY;
     int iZ = iX*iY;
}
```

- Perform the control flow analysis of the above program.

83

Static Analysis – Control Flow Analysis

What is unreachable code?

Unreachable code, a part of the source code that will never be executed due to inappropriate exit points/control flow. The other kind of unreachable code is referred as dead code, although dead code might get executed but has no effect on the functionality of the system.

Example:

Below example clearly shows unreachable code and dead code differences:

```
Function add(x, y)
{
  int z = x * y;
  return x + y;
}
```

Dead Code

```
Function add(x, y)
{
  return x + y;
  int z = x * y;
}
```

Unreachable CODE

84

Static Analysis – Control Flow Analysis

Unreachable Code Side Effects:

- Unnecessary memory overheads.
- Unnecessary caching cycles which leads to performance bottlenecks.
- Documenting and Maintaining overheads.

Unreachable Code Causes:

- Programming errors while developing complex conditional branches.
- Incomplete unit testing because of which unreachable code was undetected.
- Redundant code that developer forgot to delete
- The code that might be programmatically correct but won't be executed at any point of time due to the input data that is passed to the function.

85

Control Flow Analysis

Below are a few simple examples of unreachable code.

Example 1:

```
int func(int a)
{
   int b = a*2;
   return b;

   // Unreachable code
   if (b < 10)
   {
        b += 10;
   }
   return b;
}
```

The condition 'b<10' in this sample will never be executed, since it stands below the unconditional return operator. Since the condition 'b<10' can never be executed, the variable 'b' will also never be incremented by 10.

86

Control Flow Analysis

Example 2:

```
vector<string> vec;
while (true)
{
   vec.push_back("abc");

   if (vec.size() > 25)
   {
       return 0;
   }
}

// Unreachable code
vec.pop_back();
```

The infinite loop in this sample fills the 'vec' container with the string "abc". As the container's size exceeds 25, the return operator will be called to immediately terminate the function and bring control back to the return point. Due to this, the last item of the 'vec' container will never be removed.

87

In-Class Exercise

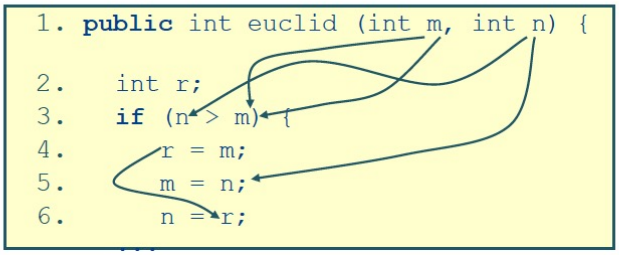
- Perform control flow analysis to the following code

```
void Tricky () {
int var1, var2, var3;
cin>> var2;
cin>> var1;
while (var2 < 10) {
   var3 = var2 + var1;
   var2 = 4;
   var1 = var2 + 1;
   cout<< var3;
    if var1 = 5
      cout<< var1;
    else
      cout<< var1+1;
   var2 = var2 + 1 ;
}
cout << var2+var1;
}
```

90

Data Flow Analysis

- Usage of data on „paths“ through the program code



- Defects cannot always be detected. Often these events can also be called data flow anomalies,
  - Reading of a variable without a prior initialising or
  - Non-usage of an assigned value of a variable

Source: MSTB/GTB v1.0

91

Data Flow Anomalies

- The usage of every single variable is analysed
- Three usages or conditions of variables can be differentiated
  - Undefined (**u**): the variable has no defined value (e.g. at the start of the program, when it is de-allocated, or when it leaves its area of validity)
  - Defined (**d**): the variable has been allocated a value
  - Referenced (**r**): the value of the variable is being read or used
- Three types of data flow anomaly are defined
  - **ur**-Anomaly: An undefined value (u) of a variable is read (r)
  - **du**-Anomaly: The variable contains a value (d), but which is made undefined (u), before it has been used
  - **dd**-Anomaly: The variable is assigned a value (d) for a second time before the first value has been used
- In summary: A data flow anomaly is the referenced usage of a variable without prior initialising non-usage of a value assigned to a variable

Source: MSTB/GTB v1.0

92

Defined Objects (d)

- An object (e.g. variable) is defined when it:
  - Appears in a data declaration.
  - Appears as parameters passed in a method header.
  - Is assigned a new value
  - Is a file that has been opened.
  - Is dynamically allocated

93

Undefined Objects (u) / `Kill`

- An object is undefined when it is:
  - Released (e.g. free) or otherwise made unavailable (eg. Out of scope).
  - A loop control variable when the loop exits
  - A file that has been closed.

94

Used/ Referenced Objects (r)

- An object is used or referenced when it is part of a computation or a predicate.
- A variable is used for a computation (c) when it appears on the RHS (sometimes even the LHS in case of array indices) of an assignment statement.
- A variable is used in predicate (p) when it appears directly in that predicate.

95

Static Analysis – Data Flow Analysis

Combination	Description	Anomaly Possibilities
dd (define-define)	Defined the data objects twice	Potential bug. Double definition.
du (define – kill / undefined)	Defined the data object but killed without using it	• Potential bug. • Data is killed without use after definition. • Bad Programming Practice
dr (define – referenced)	Defined the data object and using it	• Allowed. Normal case. • NOT an anomaly
ud (undefined/kill – define)	Killed the data object and re-define it	• Allowed. Data is killed and then re-defined. • NOT an anomaly
uu	Killed the data object and killed it	• Potential bug. • Bad Programming Practice
ur (kill/undefined – use)	Killed the data object and then use it	Serious Defect. Data is used after being killed.
rd (referenced – define)	Used the data object and re-define it	• Allowed. Data is used and then redefined. • NOT an anomaly
ru (referenced – undefined)	Used the data object and killed it	• Allowed • NOT an anomaly
rr (referenced-referenced)	Used the data object and used it again	• Allowed. Normal case. • NOT an anomaly

96

Example: Data Flow Anomalies

Example: Data Flow Anomalies

```
1.  void Swap (int min, int max) {   // d(min, max)
2.     int temp;                     // u(temp) (in Java temp=0!!)
3.     if (min > max) {              // r(min, max)
4.          max  = temp;             // d(max), r(temp)
5.          max  = min;              // d(max), r(min)
6.          temp = min;              // d(temp), r(min)
7.     }                             // u(temp)
8.  }
```

ur (temp)

du (temp)

dd (max)

Source: MSTB/GTB v1.0

97

```
1  void Swap  (int min, int max)
   {
2  int temp;
3  if (min > max){
4   max = temp;
5   max = min;
6   temp = min;
7  }}
```


98

Explanation of the Anomalies

- ur-Anomaly of the variable `temp`
  - The variable's area of validity is limited to the function. The first usage of the variable is on the right side of an allocation. The variable at that point still has an undefined value, which is being referenced at that point. An initialisation of the variable declaration has not been performed.
- dd-Anomaly of the variable `max`
  - The variable used in successive statements, each time on the left side of an allocation. This means a value allocated to it twice. Either the first allocation should be deleted or a usage of the first value has been forgotten (or, as being done here, names of variables and sequences have been mixed up).
- du-Anomaly of the variable `temp`
  - In the last statement of the function, the variable `temp` gets allocated to a value, which cannot being used anywhere, as the variable is only valid inside the function.

Source: MSTB/GTB v1.0

Evaluation of the Data Flow Analysis

- In the example, the anomalies are very obvious
- But: Between the affected statements, which lead to anomaly, there can be any amount of other statements with other variables.
  - Anomalies are then not so obvious any more
  - They can be easily overlooked when checked manually
  - Tools for data flow analysis will detect these anomalies
- Not every anomaly leads directly to defective behaviour
  - For example, a du-anomaly does not always have a direct effect. The program can run correctly
  - The question is, why is there an allocation at this point of the program, just before the end of the variables area of validity?
  - Often it is worth to arrange a deeper review of the parts of the program containing anomalies, so as to detect possible semantic defects which go deeper.

Source: MSTB/GTB v1.0

Data Flow Anomaly

- Anomaly: It is an abnormal way of doing something.
  - Example 1: The second definition of x overrides the first.

    x = f1(y);

    x = f2(z);

- Three types of abnormal situations with using variable.
  - Type 1: Defined and then defined again (dd)
  - Type 2: Undefined but referenced (ur)
  - Type 3: Defined but not referenced (du)

101

Data Flow Anomaly

- Type 1: Defined and then defined again
  - Four interpretations of Example 1
    - The first statement is redundant.
    - The first statement has a fault -- the intended one might be: w = f1(y).
    - The second statement has a fault – the intended one might be: v = f2(z).
    - There is a missing statement in between the two: v = f3(x).
  - Note: It is for the programmer to make the desired interpretation.
- Type 2: Undefined but referenced
  - Example: x = x – y – w; /* w has not been defined by the programmer. */
  - Two interpretations
    - The programmer made a mistake in using w.
    - The programmer wants to use the compiler assigned value of w.
- Type 3: Defined but not referenced
  - Example: Consider x = f(x, y). If x is not used subsequently, we have a Type 3 anomaly.

102

In-Class Exercise

- Perform data flow analysis to the following code

```
void CalculateCommission () {
int total, number;
double commission_hi, commission_lo;
cin >> number;
 while (number ≠ -1){
        total = total + number;
        cin >> number;
  }
If (total > 1000)
    commission_hi = 100 + 0.2 * ( total – 1000 );
else
    commission_lo = 0.15 * total;
cout<< "This salesman's commission is:";
cout<<  commission_hi ;
}
```


103

```
1   import java.util.Scanner;
2   public class StaffDiscount
3   {
4       public static void main(String[] args) {
5
6       double staffDiscount, totalPrice, finalPrice, discount, price;
7       staffDiscount = 0.1;
8       Scanner input = new Scanner(System.in);
9       try{
10          price = input.nextDouble();
11          while(true){
12              totalPrice = totalPrice + price;
13              price = input.nextDouble();
14           price = 50.50;
15          }
16          throw new NullPointerException();
17      }
18      catch(ArithmeticException e){
19          System.out.println("Arithmetic Error");
20          throw new ArithmeticException();
21      }
22      System.out.print("Discount: " + discount);
23      finalPrice = totalPrice-discount;
24  }
25  }
```

104